
QUICK REFERENCE

RAG Field Guide

A Field Guide for Designing Retrieval-Augmented Generation Systems

Every RAG design decision is ultimately an evidence engineering decision

Optimize evidence before optimizing generation.

- Use this handbook as a field guide for designing, evaluating and operating Retrieval-Augmented Generation (RAG) systems.
- Focus on evidence: what each technique finds, filters, validates, costs and makes trustworthy.
- Connect implementation choices to an evidence-centric and uncertainty-centric view of RAG.

Contents

How to Use This Handbook	1
1 Core RAG Mental Model	1
1.1 Decision Workflow	2
2 RAG Architecture Landscape	2
3 Retrieval Quick Reference	3
3.1 Retrieval Decision Rules	4
4 Ranking and Fusion	4
4.1 Cost–Quality Ladder	5
5 Architecture Cards	5
6 Evaluation Metrics	6
6.1 Evaluation Protocol	7
7 Engineering Decisions	7
7.1 Common Failure Modes	8
7.2 Failure Diagnosis by Uncertainty	9
8 Production Readiness Checklist	9
9 Common RAG Anti-Patterns	10
10 Comparison Cheat Sheets	10
10.1 Architecture Selection by Query Type	11
11 Evidence-Centric / Uncertainty-Centric Framework	11
12 Research Notes and References	12
12.1 Essential References	12
12.2 Further Reading and Research Direction	13
Key Takeaways	14

How to Use This Handbook

Need	Best entry point
Architecture choice	Sections 2 and 5.
Retrieval/ranking design	Sections 3 and 4.
Evaluation plan	Section 6.
Production readiness	Sections 7 and 8.
Research framing	Section 11.

1 Core RAG Mental Model

RAG augments an LLM with external evidence. The system retrieves candidate evidence, selects and compresses it, then conditions generation on that evidence. Its purpose is not only to improve answers, but to make answers more inspectable, current and grounded.

Mental model in five seconds

Knowledge → Retrieve → Rank → Validate → Generate → Evaluate → Operate

A RAG system is an evidence pipeline before it is a generation pipeline. If evidence is missing, irrelevant, unsupported or stale, generation quality cannot reliably compensate.

Stage	Question	Main risk	Typical techniques
Acquire	What knowledge is acquired?	Incomplete or stale corpus	Parsing, metadata, chunking, enrichment.
Retrieve	What evidence is found?	Search uncertainty	BM25, dense, hybrid, graph retrieval.
Rank	What evidence is prioritized?	Relevance uncertainty	Bi-encoder, cross-encoder, ColBERT, RRF.
Validate	Is evidence reliable and sufficient?	Trust and sufficiency uncertainty	CRAG, Self-RAG, entailment checks, citation checks.
Generate	How is the answer produced?	Hallucination and unsupported claims	Grounded prompting, citation constraints, structured outputs.
Operate	Does it work under constraints?	Cost, latency and drift	Caching, observability, eval sets, feedback loops.

Core operating principle

Minimal pipeline: User query → retrieval → reranking → context assembly → generation → citation/validation → answer.

Design principle: Do not choose a RAG architecture by fashion. Choose it by the uncertainty bottleneck: search, coverage, relevance, trust, sufficiency or action.

RAG in context

RAG is most useful when answers must be grounded in external, changing or auditable evidence. It complements, but does not replace, fine-tuning, long-context prompting or tool use.

Use RAG when knowledge changes often, citations matter, source permissions matter or the corpus is too large to fit reliably in context. **Consider fine-tuning** when the problem is style, format or task behavior. **Use tools/agents** when the system must calculate, modify state or execute workflows.

Golden rule

Optimize evidence before optimizing generation. Most RAG failures are not caused by weak prose; they are caused by missing, noisy, stale, insufficient or unverified evidence.

Citation warning

Citation is not the same as faithfulness. A generated answer can cite a retrieved passage and still make a claim that the passage does not actually support. Evaluate claim-level support, not only whether citations are present.

Claim-level verification prompt:

For each claim, return JSON: {"claim": str, "evidence_id": str, "support": "supported|partial|unsupported", "reason": str}.

Only mark supported when the cited evidence entails the claim.

1.1 Decision Workflow

If the query needs...	Start with...	Why
Exact identifiers, API names, error strings or legal clauses	BM25 + metadata filters + rerank	Lexical evidence is the bottleneck; dense retrieval may miss exact constraints.
Paraphrased conceptual support	Dense or Hybrid RAG	Semantic matching matters, but hybrid protects against terminology mismatch.
Multi-document synthesis	Hybrid + RRF + cross-encoder + compression	Broad recall plus clean context is more important than one perfect passage.
Corpus-level or relationship-heavy reasoning	GraphRAG, RAPTOR or hierarchical retrieval	The bottleneck is coverage and abstraction across entities, sections or communities.
High-stakes factual answering	Hybrid + verification, CRAG or Self-RAG	Trust and sufficiency must be checked before final generation.
Heterogeneous production traffic	Adaptive-RAG	Simple queries should remain cheap; complex queries need deeper retrieval.
Actions, tools or iterative research	Agentic RAG	The bottleneck is deciding what to do next, not only retrieving context.

When RAG is not enough

Do not expect RAG alone to solve every knowledge problem. Add other mechanisms when the answer requires one of the following:

- **Missing evidence:** the corpus does not contain the answer, so retrieval cannot recover it.
- **Computation or state:** the answer requires calculation, database updates, tool calls or live system state.
- **Contradictory evidence:** sources disagree and the system must reason about authority, recency or policy.
- **Judgement under uncertainty:** the user needs expert interpretation, not only grounded extraction.
- **Policy or safety decisions:** the system must abstain, escalate or enforce constraints beyond generation.

2 RAG Architecture Landscape

Family	What it adds	Uncertainty reduced	Extra components	Cost	Use cases	Quick judgement
Naive RAG	Basic retrieve-then-generate	Search	Retriever + vector DB	Low	FAQs, simple QA	Fast baseline; weak validation.
Advanced RAG	Query rewriting, reranking, compression	Search + relevance	Rewriter, reranker, compressor	Medium	Enterprise QA	Good default architecture.
Hybrid RAG	Sparse + dense retrieval	Search + coverage	BM25 + dense + fusion	Low-Med	Mixed corpora	Strong general-purpose recall.
GraphRAG	Graph, entity and community retrieval	Coverage + relation	KG/entity graph, community summaries	High	Global and relation-heavy questions	Powerful but expensive to build and maintain.
CRAG	Retrieval evaluator and correction	Trust	Evaluator, retry/search, filter	Medium	High-risk factual QA	Good when retrieval may be wrong.

Family	What it adds	Uncertainty reduced	Extra components	Cost	Use cases	Quick judgement
Adaptive-RAG	Routes by query complexity	Sufficiency + cost	Complexity classifier/router	Medium	Production QA at scale	Balances cost and quality.
Self-RAG	Model retrieves/critiques on demand	Trust + sufficiency	Reflection/critique tokens or judge	High	Long-form factual answers	Quality gain with inference/training cost.
Agentic RAG	Planning, tools, iterative retrieval	Action	Planner, tools, memory, verifier	Very high	Workflows, research agents	Most flexible; hardest to control.
RAPTOR	Tree of recursive summaries	Coverage + abstraction	Clustering, summaries, retrieval tree	High offline	Long documents	Good for multi-level context.
Hierarchical RAG	Multi-level document retrieval	Coverage	Doc/section/chunk indexes	Medium	Technical docs, policies	Useful with structured corpora.

Default recommendation

Start with **Advanced Hybrid RAG**: BM25 + dense retrieval, RRF fusion, cross-encoder reranking, metadata filters, context compression and grounded generation. Add CRAG, Self-RAG or Agentic mechanisms only when a specific failure mode justifies the extra cost.

Choosing your first production RAG

1. Start with hybrid retrieval, not agentic complexity.
2. Use BM25 for exact terms, dense retrieval for semantics and RRF to fuse them.
3. Add a cross-encoder when top-k relevance affects correctness.
4. Add metadata filters before retrieval for permissions, time and hard constraints.
5. Compress context only after retrieval and reranking, not before.
6. Require citations, but verify claim-level support separately.
7. Add CRAG-style checks when evidence can be stale, noisy or wrong.
8. Add Adaptive-RAG when production traffic has mixed complexity and cost pressure.
9. Add Agentic RAG only when the task requires planning, tools or action.
10. Log evidence IDs, ranks, scores, prompts, citations, latency and cost for every answer.

Default: Hybrid → Better ranking? Cross-Encoder → Need trust? CRAG → Need cost routing? Adaptive
→ Need action? Agentic

3 Retrieval Quick Reference

Technique	Core idea	Strengths	Weaknesses	Cost	Metrics
BM25	Sparse lexical ranking	Exact terms, acronyms, IDs, names	Synonyms, paraphrases, semantic similarity	Low	Recall@k, MRR, nDCG.
Dense Retrieval	Embedding similarity	Semantic matching, paraphrases	Rare terms, exact constraints, domain drift	Low–Med	Recall@k, Hit@k, nDCG.
Hybrid Retrieval	Sparse + dense candidates	Mixed keyword/semantic needs	Tuning/fusion complexity	Low–Med	Recall@k, coverage, MRR.
RRF	Rank-level fusion	Combines rankings robustly	Ignores calibrated score magnitudes	Low	nDCG, MRR, Recall@k.
Query Expansion	Adds terms/variants	Recall for underspecified queries	May introduce noise	Low–Med	Recall@k, precision drop.

Technique	Core idea	Strengths	Weaknesses	Cost	Metrics
Multi-query	Generates several query variants	Coverage and ambiguity	More retrieval calls	Medium	Recall@k, answer coverage.
HyDE	Generates a hypothetical answer/document, then embeds it	Semantic recall for vague queries	Can bias retrieval if hypothesis is wrong	Medium	Recall@k, downstream faithfulness.
Metadata Filtering	Constrains candidate space	Precision, compliance, temporal scope	Bad metadata can hide relevant docs	Low	Precision@k, filter hit rate.
Graph Retrieval	Traverses entities/relations	Relational coverage	Graph construction and maintenance	High	Coverage, entity recall.
Web/External Retrieval	Uses live external search	Freshness and open-world gaps	Trust, latency, source quality	Med-High	Freshness, source reliability.

3.1 Retrieval Decision Rules

- Use BM25 when exact terms matter: product codes, legal references, biomedical names, logs, errors and APIs.
- Use dense retrieval when meaning matters more than wording: conceptual QA, support tickets and paraphrased questions.
- Use hybrid retrieval as the default when the corpus mixes natural language with IDs, tables, names and domain terminology.
- Use graph retrieval when the question requires traversing entities, dependencies, ownership, causality or communities.
- Use multi-query or HyDE when recall is poor because the user query is underspecified or far from document wording.
- Use metadata filters before retrieval for hard constraints; use reranking when they are soft preferences.

4 Ranking and Fusion

Method	Idea	Advantages	Limitations	Best use	Cost
Bi-Encoder	Encode query and docs independently; compare vectors	Very scalable; supports ANN	Limited query-document interaction	First-stage retrieval	Low.
Cross-Encoder	Encode query-document pair jointly	Best relevance quality	Expensive; cannot precompute doc interactions	Rerank top 20–200	High.
Late Interaction / ColBERT	Encode separately but compare token-level representations	Better quality than pure bi-encoder; faster than cross-encoder	Larger indexes; more complex infra	Retrieval or rerank with quality/latency balance	Med-High.
RRF	Fuse ranked lists with reciprocal rank	Robust; no score normalization	Parameter k and candidate depth still matter	Hybrid sparse+dense fusion	Low.
Weighted Score Fusion	Combine normalized scores	Can exploit calibrated scores	Normalization brittle across retrievers	Controlled pipelines with stable scores	Low.
LLM-as-Reranker	LLM judges relevance	Flexible, explainable criteria	Slow, costly, prompt-sensitive	Small top-k, high-value tasks	Very high.

Canonical production stack

1. Retrieve broad candidates with BM25 and dense retriever.
2. Fuse with RRF.
3. Rerank top candidates with cross-encoder or ColBERT.

4. Compress/assemble context with metadata-aware ordering.
5. Generate with citations and validation.

4.1 Cost–Quality Ladder

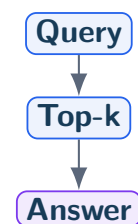
Level	Configuration	Quality	Latency/cost	When to stop
1	BM25 only	Low–Med	Very low	Corpus is keyword-heavy and queries are simple.
2	Dense only	Medium	Low–Med	Corpus is semantic and terminology is stable.
3	BM25 + dense + RRF	High recall	Medium	Good default for most systems.
4	Hybrid + cross-encoder rerank	High precision	Med–High	Use when top-k relevance affects correctness.
5	Hybrid + rerank + CRAG/Self-RAG	High trust	High	Use for critical factual answers.
6	Agentic RAG	Task success	Very high	Use for workflows, not simple QA.

5 Architecture Cards

Visual language: ■ Retrieval ■ Ranking ■ Validation ■ Generation ■ Operations

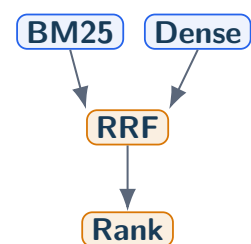
Naive RAG Low cost

Quick check: use only when one retrieved snippet is enough.
Critical metric: Hit@k / Recall@k.
Failure guard: add reranking when irrelevant context distracts generation.



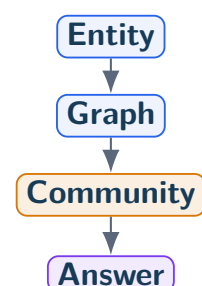
Hybrid RAG Low–medium cost

Quick check: default for mixed keyword/semantic corpora.
Critical metric: Recall@k before rerank.
Failure guard: tune lexical/dense balance and RRF depth.



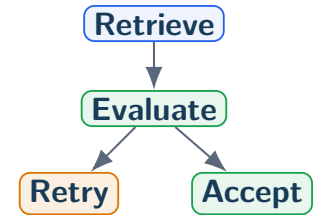
GraphRAG High offline cost

Quick check: use for entity, relation and corpus-level questions.
Critical metric: coverage / answer completeness.
Failure guard: validate entity extraction and community summaries.

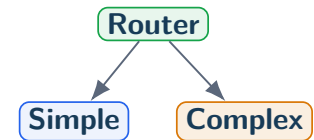


Corrective-RAG / CRAG Medium cost

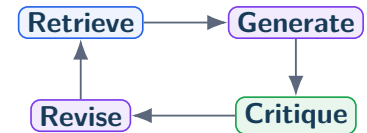
Quick check: use when retrieved evidence may be weak or stale.
Critical metric: faithfulness / citation accuracy.
Failure guard: route low-confidence evidence to retry or external search.

**Adaptive-RAG** Medium cost

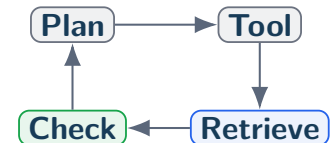
Quick check: use when query complexity varies widely.
Critical metric: routing accuracy / cost per success.
Failure guard: log route decisions and override unsafe cheap paths.

**Self-RAG** High cost

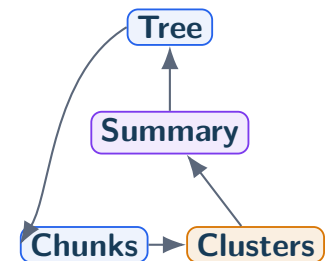
Quick check: use when answers need self-critique or abstention.
Critical metric: unsupported-claim rate.
Failure guard: cap critique loops and require evidence for revisions.

**Agentic RAG** Very high cost

Quick check: use for tasks requiring actions, tools or recovery.
Critical metric: task success / recovery rate.
Failure guard: sandbox tools, validate steps and budget iterations.

**RAPTOR / Hierarchical RAG** High offline, medium online

Quick check: use for long documents with multi-level abstraction.
Critical metric: context recall / completeness.
Failure guard: audit summaries against source chunks.



6 Evaluation Metrics

Evaluation must be separated by layer. A RAG system can have good generation quality while retrieval is poor, or good retrieval metrics while final answers remain unsupported.

Metric	Layer	What it measures	Interpretation / risk
Recall@k	Retrieval	Fraction of relevant evidence retrieved in top-k	Low recall means the generator never sees needed evidence.
Precision@k	Retrieval	Fraction of retrieved items that are relevant	Low precision wastes context and increases distraction.
Hit Rate@k	Retrieval	Whether at least one relevant item appears in top-k	Good for QA where one passage is enough.

Metric	Layer	What it measures	Interpretation / risk
MRR	Retrieval/ranking	Rank of first relevant result	Useful when the first relevant evidence matters.
MAP	Ranking	Average precision across relevant results	Useful with multiple relevant documents.
nDCG@k	Ranking	Position-aware ranking quality with graded relevance	Best general ranking metric when relevance grades exist.
Context Precision	RAG context	How much supplied context is useful	Low value means context noise.
Context Recall	RAG context	Whether context contains needed evidence	Low value means insufficient grounding.
Faithfulness	Generation	Whether claims are supported by evidence	Central hallucination-control metric.
Groundedness	Generation	Traceability of answer to retrieved sources	Requires citations or evidence mapping.
Answer Relevancy	Generation	Whether answer addresses the question	Can be high even if not faithful; evaluate separately.
Hallucination Rate	Generation	Unsupported or false claims	Must be measured with human or judge checks.
Latency p50/p95/p99	Production	Response-time distribution	Use p95/p99 for UX and SLA risk.
Cost per answer	Production	Tokens + retrieval + rerank + tools	Essential for scaling decisions.
Coverage	Production	Share of queries system can answer safely	Low coverage suggests fallback/escalation needed.
Citation accuracy	Generation/UX	Citations point to supporting evidence	Important for auditability.

6.1 Evaluation Protocol

- Create a gold set with query, expected answer, relevant evidence IDs and difficulty label.
- Evaluate retrieval before generation: Recall@k, nDCG@k and failure cases.
- Evaluate generation with evidence visible: faithfulness, groundedness, answer relevancy and citation accuracy.
- Segment results by query type: lookup, comparison, synthesis, multi-hop, temporal, procedural and open-ended.
- Report cost and latency together with quality; otherwise architecture comparisons are incomplete.
- Maintain a failure taxonomy: missing evidence, wrong evidence, irrelevant evidence, insufficient evidence, unsupported answer, bad synthesis and tool failure.

7 Engineering Decisions

Decision	Options	Main trade-off	Practical rule
Chunking	Fixed, semantic, section-aware, hierarchical	Granularity vs context coherence	Prefer structure-aware chunks when documents have headings; typical text chunks are 300–800 tokens.
Chunk size	Small, medium, large	Recall vs completeness	Start at 400–600 tokens; use 800–1,200 for narrative synthesis and 150–300 for dense facts/API docs.
Overlap	None, low, high	Boundary loss vs redundancy	Typical overlap is 10–20%; above 25% often increases duplication more than recall.
Metadata	Source, time, author, section, permissions, entity tags	Filtering power vs maintenance	Treat metadata as first-class evidence.

Decision	Options	Main trade-off	Practical rule
Embeddings	General, domain, multilingual, instruction-tuned	Generalization vs domain fit	Benchmark on your corpus; do not assume leaderboard transfer.
Vector index	HNSW, IVF, PQ, flat	Latency/memory vs recall	HNSW is a strong default; PQ for very large indexes.
Rerank depth	Top 20, 50, 100, 200	Quality vs latency	Typical: 20–50. High recall: 100. Above 100 usually adds latency with diminishing quality returns; verify saturation.
Context assembly	Top-ranked, diversified, section-ordered, source-grouped	Relevance vs coherence	Order context by source/section when synthesis matters.
Compression	Extractive, abstractive, LLM compression	Token savings vs evidence distortion	Prefer extractive compression for auditability.
Caching	Embedding, retrieval, answer, tool cache	Cost vs freshness	Cache stable layers; invalidate by corpus updates.
Security	ACL filters, source whitelists, prompt-injection checks	Safety vs recall	Apply permissions before retrieval/reranking.
Observability	Trace retrieval, rank, prompt, answer, citations	Overhead vs debuggability	Log evidence IDs and scores for every answer.

7.1 Common Failure Modes

Failure	Symptom	Likely cause	Fix
Lost answer	Correct answer not found	Poor chunking, low recall, embedding mismatch	Hybrid retrieval, query expansion, better chunks.
Distracted answer	Answer uses irrelevant context	Low precision, excessive top-k	Rerank, compress, reduce top-k.
Unsupported claim	Fluent answer not in evidence	Prompt/generation issue	Faithfulness checks, citation constraints, CRAG/Self-RAG.
Wrong citation	Citation does not support claim	Poor citation mapping	Claim-evidence verification.
Slow system	High latency	Too many retrieval/rerank/tool calls	Adaptive routing, caching, smaller top-k.
Expensive system	High token/tool cost	Over-retrieval and large contexts	Compression, routing, cheaper first-stage models.
Stale answer	Outdated evidence used	Index not refreshed	Freshness metadata, recrawl policy, external retrieval.
Access leak	User sees restricted source	Permissions after retrieval	Apply ACL pre-filtering before ranking.

7.2 Failure Diagnosis by Uncertainty

Observed failure	Likely uncertainty	Primary response
Missing relevant source	Search uncertainty	Improve chunking, hybrid retrieval, query expansion, metadata coverage and recall evaluation.
Only partial evidence retrieved	Coverage uncertainty	Add multi-query retrieval, hierarchical indexes, RAPTOR or graph/community summaries.
Correct source appears but low in ranking	Relevance uncertainty	Tune fusion depth, add cross-encoder/ColBERT reranking and evaluate nDCG/MRR.
Answer cites weak or unrelated evidence	Trust uncertainty	Add claim-evidence checks, citation verification, entailment checks and source trust scoring.
System answers when evidence is insufficient	Sufficiency uncertainty	Add abstention policy, Self-RAG/CRAG checks and fall-back/escalation behavior.
Agent loops or uses wrong tool	Action uncertainty	Constrain tools, add planner validation, step budgets, tool schemas and final-state checks.
Quality regresses after updates	Operational uncertainty	Version indexes, monitor drift, automate regression evals and log evidence IDs/scores.

8 Production Readiness Checklist

The goal of this section is to turn RAG design into an operational readiness review. Before launch, use it to verify that the corpus, retrieval layer, generation behavior, monitoring, safety controls and rollback paths are explicit enough to support a reliable production service.

Corpus readiness

- ☐ Document ownership defined
- ☐ Update cadence known
- ☐ Access permissions represented
- ☐ Parsing quality checked
- ☐ Metadata schema defined
- ☐ Deduplication strategy in place

Retrieval readiness

- ☐ Gold retrieval set available
- ☐ BM25/dense/hybrid benchmarked
- ☐ Rerank depth tuned
- ☐ Recall@k and nDCG tracked
- ☐ Failure cases reviewed manually
- ☐ Temporal and permission filters tested

Generation readiness

- ☐ Prompt grounded in evidence
- ☐ Citation format enforced
- ☐ Unsupported-answer fallback defined
- ☐ Faithfulness checked
- ☐ Context length budget defined
- ☐ Structured output validated when needed

Operations readiness

- ☐ p50/p95/p99 latency tracked
- ☐ Cost per answer tracked
- ☐ Trace logs include evidence IDs
- ☐ Regression eval suite automated
- ☐ Feedback loop defined
- ☐ Rollback/index versioning available

In production

- ☐ Monitor retrieval, ranking and answer-quality drift
- ☐ Review failed, abstained and escalated queries
- ☐ Track freshness, stale sources and index lag
- ☐ Audit citations, access control and sensitive data exposure
- ☐ Re-run regression evals before prompt, model or index changes
- ☐ Keep rollback plan and incident owner current

Safety readiness

- ☐ Prompt injection detection
- ☐ Source trust scoring
- ☐ PII/security policy
- ☐ Human escalation path

☐ Audit logs

☐ Red-team queries maintained

9 Common RAG Anti-Patterns

Anti-pattern	Why it fails	Better pattern
More context is always better	Large contexts increase distraction, cost and citation ambiguity.	Retrieve broadly, rerank aggressively, then compress or diversify context.
Dense-only retrieval by default	Exact IDs, names, legal clauses and rare terms are often lexical.	Use hybrid retrieval unless the corpus is purely semantic and stable.
Citations imply support	A citation can point to a source without supporting the generated claim.	Verify claim-level entailment and citation accuracy.
One metric is enough	Retrieval, ranking, generation and production can fail independently.	Evaluate by layer: Recall@k, nDCG, faithfulness, latency and cost.
LLM judge as ground truth	Judges are prompt-sensitive and can share model biases.	Combine human gold sets, deterministic checks and calibrated judge prompts.
Permissions after retrieval	Restricted documents can leak through ranking, logs or context assembly.	Apply ACL and policy filters before retrieval and reranking.
Agentic RAG for simple QA	Agents add latency, cost and control risk when no action is needed.	Use the simplest architecture that removes the actual uncertainty bottleneck.

Practical takeaway

RAG quality improves fastest when failures are diagnosed at the evidence layer first: did the system find, select, validate and use the right evidence?

10 Comparison Cheat Sheets

Comparison	Rule of thumb
BM25 vs Dense	BM25 is lexical and exact; dense is semantic and paraphrase-tolerant. Use hybrid when in doubt.
Bi-Encoder vs Cross-Encoder	Bi-encoder is scalable for candidate generation; cross-encoder is stronger for reranking but expensive.
Cross-Encoder vs ColBERT	Cross-encoder maximizes interaction but costs more; ColBERT preserves token-level interaction with precomputable document representations.
RRF vs Weighted Fusion	RRF is robust without score calibration; weighted fusion is better only when scores are stable and calibrated.
Hybrid RAG vs GraphRAG	Hybrid improves evidence recall; GraphRAG improves relational/global understanding.
CRAG vs Self-RAG	CRAG adds external retrieval evaluation/correction; Self-RAG makes retrieval and critique part of generation behavior.
Adaptive-RAG vs Agentic RAG	Adaptive-RAG routes QA strategies; Agentic RAG plans and acts across tools/workflows.
RAPTOR vs GraphRAG	RAPTOR organizes abstraction hierarchically; GraphRAG organizes entities/communities relationally.
Chunk compression vs Reranking	Reranking selects better evidence; compression reduces tokens after selection. They solve different bottlenecks.
More context vs Better context	More context can reduce missing evidence but increase distraction. Better ranking/selection is usually safer.

10.1 Architecture Selection by Query Type

Query type	Recommended architecture	Why
Simple fact lookup	Naive/Advanced RAG	Single evidence item often sufficient.
Exact identifier/API/legal clause	BM25 + rerank	Lexical match is critical.
Paraphrased support question	Dense or Hybrid RAG	Semantic matching required.
Multi-document synthesis	Hybrid + rerank + compression	Need broad recall and clean context.
Global corpus question	GraphRAG or hierarchical summaries	Needs corpus-level abstraction.
Multi-hop relational question	GraphRAG + rerank	Requires relationships/entities.
High-stakes factual answer	CRAG/Self-RAG + verification	Need trust and correction.
Workflow/action task	Agentic RAG	Need planning, tool use and iteration.
Cost-sensitive production QA	Adaptive-RAG	Route simple vs complex queries.

11 Evidence-Centric / Uncertainty-Centric Framework

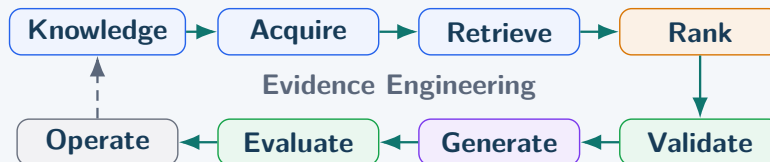
A unified view of RAG is possible if each technique is interpreted as a mechanism for reducing uncertainty around evidence. The question is not simply “which RAG architecture is best?” but “which uncertainty prevents a trustworthy answer?”

Uncertainty ladder			
Search → Coverage → Relevance → Trust → Sufficiency → Action → Operations			
Uncertainty	Techniques	Metrics	Architecture pressure
Search	BM25, dense, hybrid, HyDE	Recall@k, Hit@k	Naive/Hybrid RAG
Coverage	Multi-query, GraphRAG, RAPTOR	Context recall, completeness	GraphRAG/Hierarchical
Relevance	RRF, cross-encoder, ColBERT	nDCG, MRR	Advanced RAG
Trust	CRAG, entailment, source scoring	Faithfulness, citation accuracy	CRAG/Self-RAG
Sufficiency	Routing, abstention, self-critique	Abstention quality, success rate	Adaptive/Self-RAG
Action	Planners, tools, validators	Task success, recovery	Agentic RAG

Uncertainty dimension	Core question	Techniques that reduce it	Observable metrics
Search uncertainty	Can the system find potentially relevant evidence?	BM25, dense retrieval, hybrid retrieval, query expansion, HyDE	Recall@k, Hit@k, coverage.
Coverage uncertainty	Is the evidence complete enough?	Multi-query, GraphRAG, RAPTOR, hierarchical retrieval	Context recall, answer completeness.
Relevance uncertainty	Is the selected evidence actually useful?	Cross-encoder, ColBERT, RRF, LLM rerankers	nDCG, MRR, context precision.
Trust uncertainty	Is the evidence reliable and supportable?	CRAG, source scoring, citation verification, entailment checks	Faithfulness, groundedness, citation accuracy.
Sufficiency uncertainty	Is retrieval/generation enough for this query?	Adaptive-RAG, Self-RAG, fallback policies	Routing accuracy, abstention quality, answer success.
Action uncertainty	What should the system do next?	Agentic RAG, planners, tools, validators	Task success, tool efficiency, error recovery.

Uncertainty dimension	Core question	Techniques that reduce it	Observable metrics
Operational uncertainty	Can the system sustain quality in production?	Monitoring, caching, eval suites, drift checks	Latency, cost, SLA, regression rate.

Evidence lifecycle



Research framing: RAG variants are not isolated inventions. They are design responses to different uncertainty bottlenecks in evidence acquisition, selection, validation, sufficiency assessment and action orchestration.

12 Research Notes and References

This section links the handbook with a broader Knowledge-Centric AI line: RAG as evidence acquisition; OKF/CKB as knowledge packaging and materialization; MCP/tooling as execution substrate for knowledge consumption.

Concept	Role in the broader line
Evidence-Centric RAG	Frames retrieval and generation around evidence quality, sufficiency and traceability.
Uncertainty-Centric Framework	Organizes architectures by the uncertainty dimension they reduce.
Knowledge-Centric AI	Moves from prompt/context tricks to explicit knowledge assets, lifecycle and governance.
OKF / LLM-Wiki	Represents structured knowledge packages/bounds for LLM consumption and maintenance.
CKB	Context Knowledge Bundle: portable/executable unit of relevant context/evidence for a consumer.
MCP integration	Provides tools/resources/prompts for external consumers such as coding agents or copilots.
Knowledge materialization	Transforms abstract knowledge into consumable execution artifacts: memory payloads, structured messages, bundles or workspaces.

12.1 Essential References

Reference	Title	Why it matters	Locator
Lewis et al., 2020	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Foundational RAG formulation combining parametric and non-parametric memory.	arXiv:2005.11401
Karpukhin et al., 2020	Dense Passage Retrieval for Open-Domain QA	Dense bi-encoder retrieval baseline for QA.	arXiv:2004.04906
Khattab & Zaharia, 2020	ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT	Late interaction ranking/retrieval.	arXiv:2004.12832
Cormack et al., 2009	Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods	Simple robust rank fusion method.	SIGIR 2009
Asai et al., 2023/2024	Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection	On-demand retrieval and self-critique for factuality.	arXiv:2310.11511 / ICLR 2024
Yan et al., 2024	Corrective Retrieval Augmented Generation	Retrieval evaluator and corrective retrieval actions.	arXiv:2401.15884

Reference	Title	Why it matters	Locator
Jeong et al., 2024	Adaptive-RAG: Learning to Adapt Retrieval-Augmented LLMs through Question Complexity	Routing strategies by question complexity.	arXiv:2403.14403 / NAACL 2024
Edge et al., 2024/2025	From Local to Global: A Graph RAG Approach to Query-Focused Summarization	Graph/community-summary approach to global corpus questions.	arXiv:2404.16130
Sarathi et al., 2024	RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval	Recursive summaries/tree retrieval for long-context abstraction.	ICLR 2024
Gao et al., 2023	Retrieval-Augmented Generation for Large Language Models: A Survey	Survey organizing Naive, Advanced and Modular RAG.	arXiv:2312.10997

12.2 Further Reading and Research Direction

Reference	Title	Connection to this guide	Role
Garralda-Barrio, 2026	Knowledge-Centric Information Systems	Generalizes this guide from RAG evidence pipelines to knowledge architecture: ingestion, provenance, catalogs, curated knowledge layers and operational knowledge delivery.	Architectural foundation.
Garralda-Barrio, 2026	Governed Evolution of Agent Runtimes through Executable Operational Cognition	Extends evidence consumption into governed agent runtimes where retrieved knowledge, tools, prompts, evaluators and executable artifacts become auditable operational capabilities.	Agentic operations frame.

Key Takeaways

RAG in one page

- Retrieval is evidence acquisition.
- Ranking is evidence prioritization.
- Validation creates trust.
- Generation cannot compensate for poor evidence.
- Every RAG architecture reduces a different uncertainty.

Confidence is not assumed. It is constructed from evidence.